Scalability & Resilience in a Cloud-Native Microservice

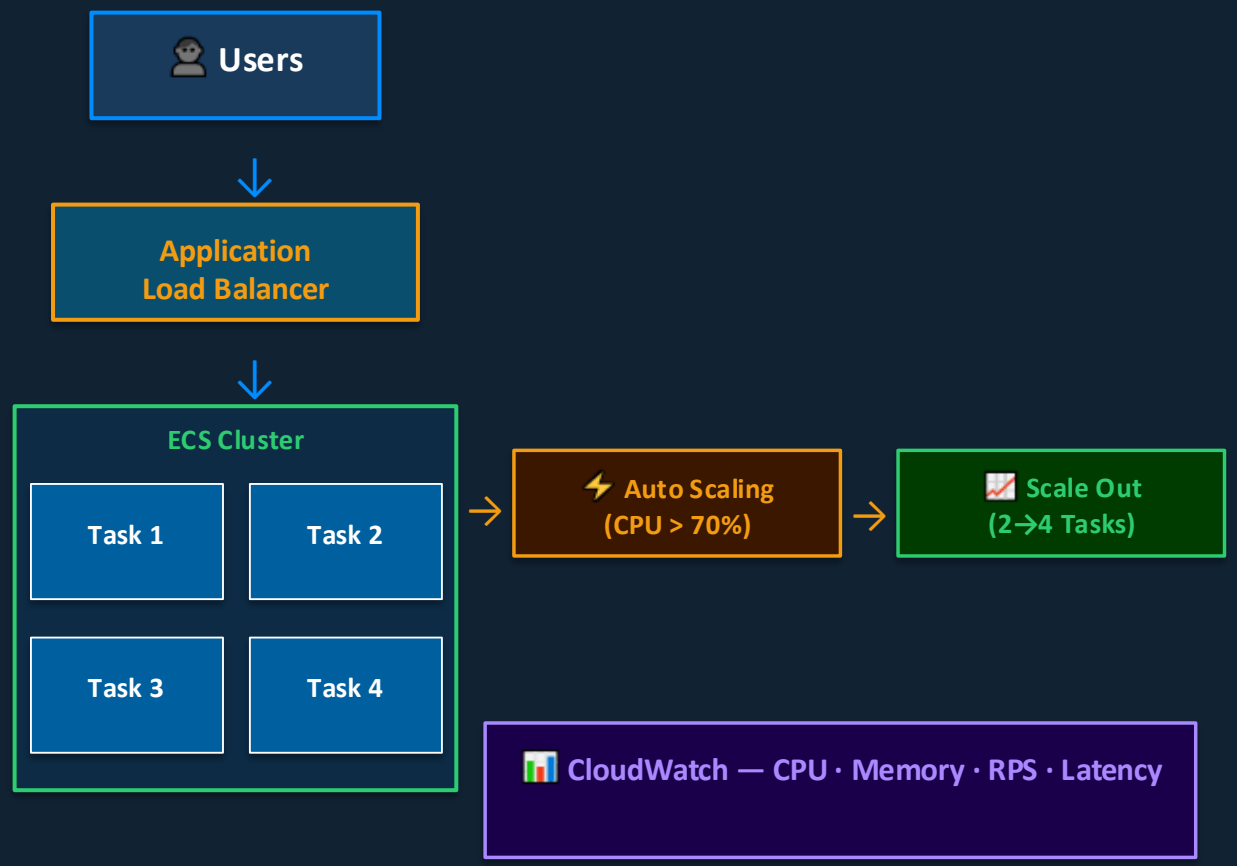
Mayank Tamakuwala

CS 6650 – Building Scalable Distributed Systems

March 2026

AWS ECS Fargate · ALB · CloudWatch · Locust

System Architecture



Components

- ECS Fargate - serverless containers (no EC2 management)
- Application Load Balancer (distributes traffic across tasks)
- Auto Scaling - CPU target 70% (scales 2 → 4 tasks under load)
- CloudWatch - real-time metrics (CPU, Memory, RPS, Latency)
- Locust load testing (progressive: 500 → 3000 users)

Stress Testing Setup

Test Parameters

- Progressive load ramp-up strategy
- Simulated users: 500 → 3,000
- Requests per second: 50 → 200
- Spawn rate: 10 users/second
- Metrics observed:
  - CPU utilization
  - Memory usage
  - Requests per second (RPS)
  - Response latency (avg & p95)
  - Failure count & rate
- Tool: Locust (Python-based)

Locust Test Report

Download the Report

During: 2/21/2026, 2:58:50 PM - 2/21/2026, 3:01:51 PM (3 minutes and 1 second)
Target Host: http://44.248.141.155:8080
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /health	35507	0	432.09	80	2091	15	196.76	0
GET	GET /products/search	358225	0	431.52	78	2371	985.7	1985.06	0
	Aggregated	393732	0	431.57	78	2371	898.16	2181.82	0

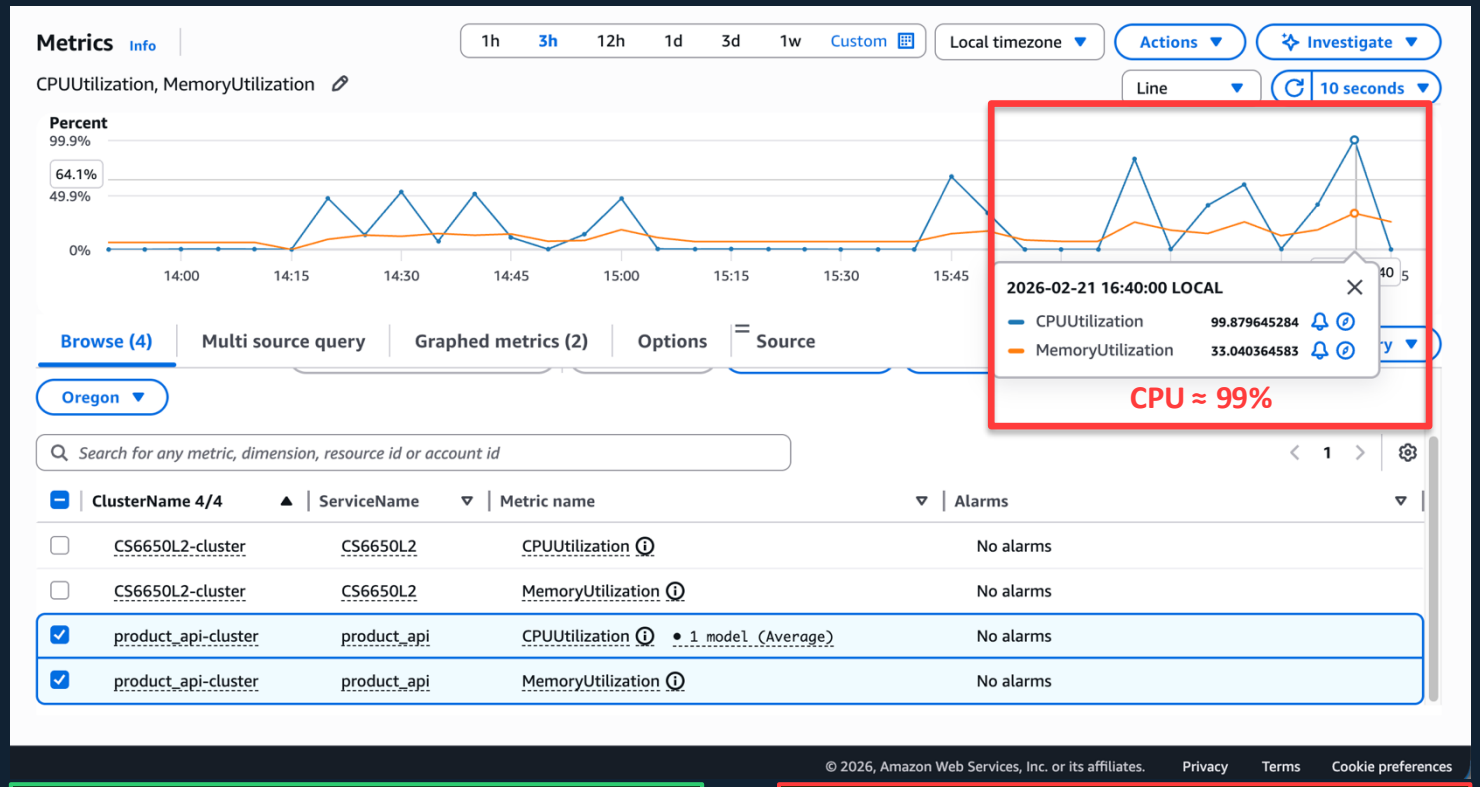
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /health	380	390	490	640	770	830	1000	2100
GET	GET /products/search	380	390	490	630	770	840	1000	2400
	Aggregated	380	390	490	630	770	840	1000	2400

Failures Statistics

# Failures	Method	Name	Message

CPU-Bound Bottleneck



Memory

~30% ✓ Stable

Throughput

Plateau ✗ Flat

Observation:

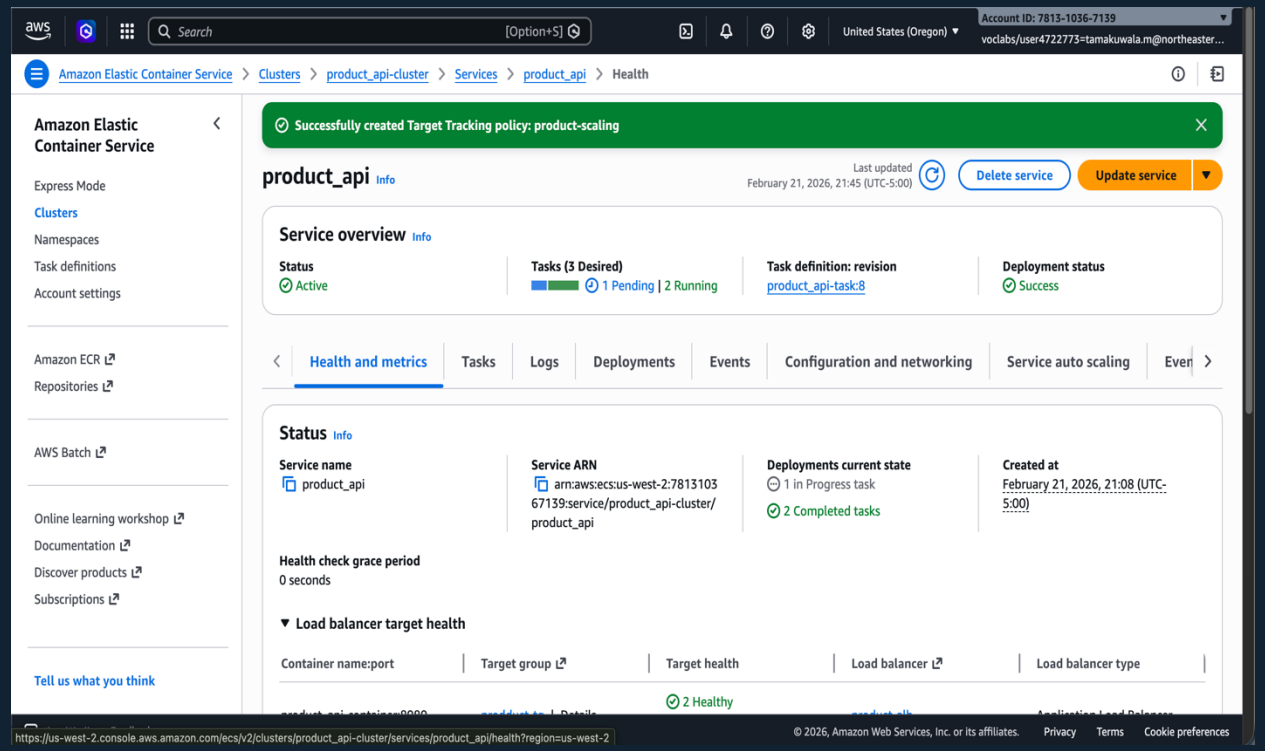
- CPU saturated at ~99% under heavy load
- Memory remained stable (~30%) - NOT the bottleneck
- Throughput plateau: RPS stops growing despite more simulated users
- Latency climbed sharply as CPU queues built up

Diagnosis:

The service is CPU-bound.

Solution → Horizontal Scaling

Auto Scaling in Action



What Happened:

- Under load, CPU crossed 70% threshold (Auto Scaling policy)
- ECS scaled out:
  Desired count 2 → 3
  Running tasks 2 → 3
- Load distributed across 3 containers
- CPU per task decreased after new task launched
- Elasticity proven: **2 Tasks → 3 Tasks**
  System adapts automatically without manual intervention

Resilience Test: Self-healing Under Task Failure

Scenario

- Service running under sustained load
- Multiple ECS tasks active behind ALB

Failure Injection

- Manually terminated one running ECS task
- Simulated unexpected container crash

System Behavior

- ECS detected stopped task automatically
- Replacement task launched to maintain desired count
- ALB redistributed traffic to healthy targets
- No service outage observed

Key Insight

- Container orchestration enables automatic recovery
- System remains available despite instance-level failure

Downstream Dependency Failure

fakeExternalService (Broken Mode)

```go
func fakeExternalService() error {
    // 30% chance of slow response
    if rand.Float64() < 0.30 {
        // Simulate downstream delay
        time.Sleep(2 * time.Second)
        return nil
    }
    return nil
}

// Called in every request handler:
if err := fakeExternalService(); err != nil
{
    http.Error(w, err.Error(), 500)
}
```

- 30% of requests hit a 2-second sleep
- No timeout cap → goroutines pile up
- Resources exhausted under load

Effect:

- 30% of requests delayed by 2s
- Cascading latency spike
- p95 latency becomes unpredictable
- System appears slow even when CPU load is moderate

Root cause: unbounded wait on a slow downstream service

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /health	24232	0	100.26	77	725	15	134.55	0
GET	GET /products/search	242399	71889	704.32	87	2728	701.49	1345.98	399.18
	Aggregated	266631	71889	649.42	77	2728	639.1	1480.53	399.18

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /health	95	97	98	100	100	120	220	730
GET	GET /products/search	110	110	330	2100	2100	2100	2200	2700
	Aggregated	110	110	170	2100	2100	2100	2200	2700

Failures Statistics

# Failures	Method	Name	Message
71889	GET	GET /products/search	CatchResponseError('Expected 200, got 503')

Applying Fail-Fast Resilience Pattern

fakeExternalService → With Timeout (Fixed)

```go
func fakeExternalServiceWithTimeout(
    timeout time.Duration) error {

    done := make(chan error, 1)
    go func() {
        time.Sleep(2 * time.Second)
        done <- nil
    }()

    select {
    case err := <-done:
        return err
    case <-time.After(timeout):
        return errors.New("timeout")
    }
}

// Called with 200ms cap:
fakeExternalServiceWithTimeout(200 *
time.Millisecond)
```

Why Fail-Fast Works:

- Caps waiting time at 200 ms
- Prevents goroutine pile-up
- Returns HTTP 503 immediately instead of timing out the client
- Frees CPU/memory for healthy reqs
- Latency becomes predictable

Broken Mode

RPS
134.55
1345.98
1480.53

→

Fixed Mode

RPS
358.24
3617.61
3975.85

Before vs After - Quantitative Comparison

Metric	Broken Mode	Fixed Mode
Avg Latency	High (2+ seconds)	Low (<300 ms)
p95 Latency	Very High (4–6s+)	Controlled (<500 ms)
RPS	Drops / unstable	Stable & consistent
Error Type	Slow client timeout	Fast HTTP 503
Resource Use	Goroutine pile-up	Goroutines freed fast
UX Impact	Long waiting / hung	Quick error, retry ok

Analytical outcome: Fail-Fast pattern eliminates cascading slowdowns while preserving system throughput.

Key Takeaways

Metrics-Driven Diagnosis

Use CPU, memory, RPS, latency together — not in isolation.

CPU-Bound Bottleneck Identified

Memory stable; only CPU saturated → pinpoint root cause.

Horizontal Scaling = Elasticity

ECS auto-scaling absorbed load increases automatically.

Self-Healing Containers

ECS restarted crashed tasks with no manual intervention.

Fail-Fast Prevents Cascades

200 ms timeout cap stopped slow dependency from starving resources.

Cloud-Native Design Pays Off

Managed services (Fargate, ALB, CloudWatch) reduce ops burden.

"Scalability is not just adding resources — it's designing for failure."